

Trust Region Policy Optimization 2017

梗概

在作者编写这篇文章的时候认为现有的 **Policy Gradient** 存在一些问题。

作者回顾了三类主流的强化学习方法

1. Policy Iteration
2. Policy Gradient
3. Derivative-Free Optimization (CEM,CMA)

作者发现，在很多实际控制任务中，人们更倾向于使用 CEM,CMA 这种随机搜索方法

因为这些方法实现简单，调参简单，效果稳定，在一些经典问题上随机搜索还经常获胜

但是，作者指出 **理论上梯度算法应该比这些随机搜索方法更强**，随机搜索的本质是，试一组参数看奖励然后再试下一组参数，这种策略完全不知道目标函数的局部结构。而 Policy Gradient $\nabla_{\theta} \eta(\theta)$ 知道梯度的方向，效率应该远高于随机搜索。因此，作者认为现有梯度算法有问题。

深度网络需要稳定的优化器。过去的很多 CMA,CEM 还能工作是因为参数较小，如果使用神经网络，参数极大，随机搜索空间太大，就开始崩溃。

综上作者提出了 **Trust Region Policy Optimization (TRPO)**

问题定义

考虑一个 Markov decision process MDP

$$(S, A, P, r, \rho_0)$$

和 Sutton 给出的定义完全一样

- S 状态空间 有限集
- A 动作空间 有限集
- P 转移概率
- r 奖励函数
- ρ_0 初始状态分布

同样的，如同许多讨论 Policy 的论文，作者将 Policy 定义为

$$\pi(a|s)$$

作者期望研究的是**策略分布变化**，而不单是动作变化

定义奖励 **Expected Discounted Return** 即策略 π 的性能

$$\eta(\pi) = E_{s_0, a_0, \dots} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t) \right] \quad (1)$$

where $s_0 \sim \rho_0(s_0)$, $a_t \sim \pi(a_t | s_t)$, $s_{t+1} \sim P(s_{t+1} | s_t, a_t)$.

定义了 **state-value function** Q_π , **value function** V_π , **advantage function** A_π

$$Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \dots} \left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l}) \right] \quad (2)$$

$$V_\pi(s_t) = \mathbb{E}_{a_t, s_{t+1}, \dots} \left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l}) \right] \quad (3)$$

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s) \quad (4)$$

where

$$a_t \sim \pi(a_t | s_t), \quad s_{t+1} \sim P(s_{t+1} | s_t, a_t), \quad t \geq 0.$$

作者提出 新策略性能 = 原策略性能 + advantage

$$\eta(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{s_0, a_0, \dots \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A_\pi(s_t, a_t) \right] \quad (5)$$

$s_0, a_0, \dots \sim \tilde{\pi}$ 这一串表示轨迹 $(s_0, a_0, \dots)$ 是由 $\tilde{\pi}$ 生成的。

① Note

为什么累计的 advantage 函数是 A_π 而不是 $A_{\tilde{\pi}}$?

如果我们考虑新策略到底比旧策略好多少，自然的想法就是用旧策略作为参照，回到 **公式 (4)**，新策略在状态 s 选择了动作 a 后的得分 $Q_\pi(s, a)$ ，然后再减去旧策略在状态 s 选择任意动作的平均得分 $V_\pi(s)$ 得到了新策略到底比旧策略好多少的 A_π 。

作者还引入了 **折扣状态访问频率 discounted state visitation frequency**

它表示 在策略 π 下，一个状态 s 被访问的加权次数

$$\begin{aligned} \rho_\pi(s) &= \sum_{t=0}^{\infty} \gamma^t P(s_t = s) \\ &= \gamma^0 P(s_0 = s) + \gamma^1 P(s_1 = s) + \gamma^2 P(s_2 = s) + \dots \end{aligned} \quad (6)$$

作者从 **公式 (5)** 改写，因为原式中还有期望，缺点是只能在计算完所有轨迹后才能计算整体期望，不利于优化，因此将期望展开成 状态概率 $\times$ 动作概率 $\times$ advantage 的形式，再通过 **公式 (6)** 的代换。

$$\begin{aligned}\eta(\tilde{\pi}) &= \eta(\pi) + \sum_{t=0}^{\infty} \sum_s P(s_t = s \mid \tilde{\pi}) \sum_a \tilde{\pi}(a|s) \gamma^t A_{\pi}(s, a) \\ &= \eta(\pi) + \sum_s \sum_{t=0}^{\infty} \gamma^t P(s_t = s \mid \tilde{\pi}) \sum_a \tilde{\pi}(a|s) A_{\pi}(s, a) \\ &= \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a|s) A_{\pi}(s, a).\end{aligned}\tag{7}$$

这样新策略比旧策略好多少可以分解成三部分

- $\rho_{\tilde{\pi}}(s)$ 新策略经常访问哪些状态
- $\tilde{\pi}(a|s)$ 新策略喜欢选择哪些动作
- $A_{\pi}(s, a)$ 这些动作在旧策略看来到底是好是坏

但是，作者指出 **公式 (7)** 虽然正确但是没法直接优化，我们不可能像 Exact Policy Iteration 那样精确的知道 $A_{\pi}(s, a)$ 因为 advantage 是估计出来的，value function 也是估计出来的，policy 也是参数化近似

所以我们没法保证 $\sum_a \tilde{\pi}(a|s) A_{\pi}(s, a) \geq 0$ 从而让策略 $\tilde{\pi}$ 总是更优的。

而且，真正困难的是 $\rho_{\tilde{\pi}}(s)$ ，因为要计算该值需要还未被求出来的 $\tilde{\pi}$ 策略，从而出现循环依赖。因此作者做出了一个小改动，将 **公式 (7)** $\tilde{\pi}$ 替换成了 π

$$L_{\pi}(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\pi}(s) \sum_a \tilde{\pi}(a|s) A_{\pi}(s, a).\tag{8}$$

这个近似替换的理论是建立在 $\tilde{\pi} \approx \pi \Rightarrow \rho_{\tilde{\pi}}(s) \approx \rho_{\pi}(s)$ 的基础上

作者认为 **策略几乎不变，访问到的状态也不会突然发生巨大变化，当前位置的梯度方向（一阶）也一样**

这就是作者提到的 **local approximate**

数学思维敏捷且严谨的同学就要问了：小步更新可以保证近似成立，有没有严格证明？

有的兄弟，有的。

作者引入了一个前人的工作 **Conservative Policy Iteration (CPI)** Kakade & Langford

定义当前策略 π_{old} 然后直接寻找

$$\pi' = \arg \max_{\pi'} L_{\pi_{old}}(\pi')$$

π' 就是在替代目标 (surrogate objective) 上最优的策略

于是在旧策略和改进策略上做加权平均

$$\pi_{new}(a|s) = (1 - \alpha)\pi_{old}(a|s) + \alpha\pi'(a|s)$$

然后引入 Kakade & Langford 的结论：真实目标 $\geq$ 替代目标 - penalty

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{2\epsilon\gamma}{(1-\gamma)^2}\alpha^2$$

这样我们得到了 η 的下界

where $\epsilon = \max_s |E_{a \sim \pi'(a|s)} [A_\pi(s, a)]|$

ϵ 表示单个状态下能取得的最大优势

这样我们就能对小步的变化幅度有了明确的数学定义

但是作者指出 CPI 仍有缺陷，这个理论只适用于

$$\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi'$$

这种特殊形式，现代神经网络更新参数时并不会自动的用这种方式更新策略，因此作者期望把这个 lower bound 推广到任意参数化随机策略

单调改进的下界

monotonic improvement guarantee

作者提出了一个距离去衡量新策略和旧策略的效能差距

首先作者引入了一个概念，取名为 **Total Variation Divergence**

$$D_{TV}(p||q) = \frac{1}{2} \sum_i |p_i - q_i|$$

where p,q 是两个概率分布，这个 TV Divergence 用来形容两个概率分布的差距

有了这个工具之后我们就能通过概率分布来定义两个策略之间的差距了

$$D_{TV}^{\max}(\pi, \tilde{\pi}) = \max_s D_{TV}(\pi(\cdot|s) \parallel \tilde{\pi}(\cdot|s)).$$

Theorem 1

当 $\alpha = D_{TV}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$ 下面的边界就成立

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2.$$

$$\text{where } \epsilon = \max_{s,a} |A_\pi(s, a)|.$$

作者在文中其实小跳了一步，借助 **Pinsker's Inequality**

$$D_{TV}(p, q)^2 \leq D_{KL}(p||q)$$

于是我们可以将 Theorem 1的不等式替换为

$$\eta(\tilde{\pi}) \geq L_{\pi}(\tilde{\pi}) - CD_{KL}^{\max}(\pi, \tilde{\pi}), \quad C = \frac{4\epsilon\gamma}{(1-\gamma)^2}.$$

作者重新将不等式右边定义为

$$M_i(\pi) = L_{\pi_i}(\pi) - CD_{KL}^{\max}(\pi_i, \pi)$$

在 $i+1$ 的时候有

$$\eta(\pi_{i+1}) \geq M_i(\pi_{i+1})$$

在旧策略的时候取等号，因为都使用旧策略的时候不存在误差

$$\eta(\pi_i) = M_i(\pi_i)$$

那么自然就有

$$\begin{aligned} \eta(\pi_{i+1}) - \eta(\pi_i) &\geq M_i(\pi_{i+1}) - M_i(\pi_i) \\ M_i(\pi_{i+1}) &\geq M_i(\pi_i) = \eta(\pi_i) \end{aligned}$$

所以我们就确定

$$\eta(\pi_{i+1}) \geq M_i(\pi_{i+1}) \geq \eta(\pi_i)$$

即，我们总能保证我们的优化总能使下一个策略不差于当前策略

Policy 参数优化

在经过上面数学的层层推导后，你可能觉得完美！可以收工了！作者却说，停停，还有一些参数的问题没有解决。

我们已经可以对策略进行不差于旧策略的迭代了，而对于神经网络来说，设参数为 θ ，优化的目标写作

$$\max_{\theta} [L_{\theta_{\text{old}}}(\theta) - CD_{KL}^{\max}(\theta_{\text{old}}, \theta)]. \quad (9)$$

经常阅读强化学习论文的朋友都知道， $\min$ ， $\max$ ，期望 这些聚类函数总是要考虑所有状态，显然对于无限状态空间是不可求的，因此，作者进一步近似，使用旧策略访问分布下的平均 KL

$$\bar{D}_{KL}(\theta_1, \theta_2) := E_{s \sim \rho} [D_{KL}(\pi_{\theta_1}(\cdot|s) \parallel \pi_{\theta_2}(\cdot|s))].$$

① Note

为什么这里用期望就比最大值好计算呢？

因为这里的 KL 已经变成一个离散的分布了

根据 Monte Carlo 的思想，可以通过该分布的采样来估计这个期望

$$\hat{D}_{KL} = \frac{1}{N} \sum_{i=1}^N E_{s \sim \rho_{\text{old}}} [D_{KL}(s_i)]$$

样本越多近似越准，而且是无偏近似，数学上这个均值的计算容易的多

除上述 KL 计算之外，在实践中，如果真的使用理论给出的 C ，这个常数非常非常保守，理论上安全但是小到不适合训练

因此，作者将 penalty 形式改写成 constraint 形式

作者不直接优化 **公式 (9)** 而是改成

$$\begin{aligned} \max_{\theta} \quad & L_{\theta_{\text{old}}}(\theta) \\ \text{subject to} \quad & D_{KL}^{\max}(\theta_{\text{old}}, \theta) \leq \delta. \end{aligned}$$

类似 **公式 (7)** 的转化，现在的优化问题可以改写为

$$\begin{aligned} \max_{\theta} \quad & \sum_s \rho_{\theta_{\text{old}}}(s) \sum_a \pi_{\theta}(a|s) A_{\theta_{\text{old}}}(s, a) \\ \text{subject to} \quad & \bar{D}_{KL}(\theta_{\text{old}}, \theta) \leq \delta. \end{aligned} \tag{10}$$

与 **公式 (7)** 有类似的问题

- **状态分布** $\rho_{\theta_{\text{old}}}(s)$
表示旧策略访问状态的频率，而状态空间可能巨大且连续，不可能把所有状态枚举出来
- **动作求和**
动作状态多的时候计算量太大
- **Advantage**
真实值不知道，只能通过估计

因此 **公式 (10)** 还是不利于计算，于是，作者进行了三步优化

1. 状态求和改写为分布期望

原来要对所有状态求和

$$\sum_s \rho_{\theta_{\text{old}}}(s) [\dots]$$

而这个 ρ 本身就是状态分布，所以作者讲求和写成期望形式

$$E_{s \sim \rho_{\text{old}}} \left[\sum_a \pi_{\theta}(a|s) A_{\text{old}}(s, a) \right]$$

2. Advantage 替换成 Q-value 根据 **公式 (4)**

$$\begin{aligned}
\sum_a \pi_\theta(a|s) A_\pi(s, a) &= \sum_a \pi_\theta(a|s) (Q_\pi(s, a) - V_\pi(s)) && \text{代入 } A_\pi = Q_\pi - V_\pi \\
&= \sum_a \pi_\theta(a|s) Q_\pi(s, a) - V_\pi(s) \sum_a \pi_\theta(a|s) && \text{提取 } V_\pi(s) \\
&= \sum_a \pi_\theta(a|s) Q_\pi(s, a) - V_\pi(s) && \text{利用 } \sum_a \pi_\theta(a|s) = 1
\end{aligned}$$

$V(s)$ 与待优化参数无关

因此

$$\begin{aligned}
&\arg \max_{\theta} \sum_a \pi_\theta(a|s) A(s, a) \\
&\arg \max_{\theta} \sum_a \pi_\theta(a|s) Q(s, a)
\end{aligned} \tag{11}$$

完全等价，只是差了一个常数

3. 重要性采样

现在的目标变成了 **公式 (11)** 但是我们手里只有旧策略采样的数据

$$a \sim \pi_{old}$$

而目标里面需要，新策略产生的数据

$$a \sim \pi_\theta$$

这两个分布不同怎么办？

作者采用了 **重要性采样 Importance Sampling**

$$\sum_a \pi_\theta(a|s_n) A_{\theta_{old}}(s_n, a) = E_{a \sim q} \left[\frac{\pi_\theta(a|s_n)}{q(a|s_n)} A_{\theta_{old}}(s_n, a) \right].$$

这里的 q 就是 π_{old}

(顺带一提这里的 $\frac{\pi_\theta(a|s_n)}{q(a|s_n)}$ 就是后来 PPO 里著名的 概率比率 $r_t(\theta)$)

Finally, 我们终于获得了一个可以被轻松计算（并非）的目标优化问题

$$\begin{aligned}
&\max_{\theta} \quad E_{s_n \sim \rho_{\theta_{old}}, a_n \sim q} \left[\frac{\pi_\theta(a|s)}{q(a|s)} Q_{\theta_{old}}(s, a) \right] \\
&\text{subject to} \quad E_{s_n \sim \rho_{\theta_{old}}} [D_{KL}(\pi_{\theta_{old}}(\cdot|s) \parallel \pi_\theta(\cdot|s))] \leq \delta.
\end{aligned} \tag{12}$$

Vine 采样

既然作者已经推导出了通过采样近似的优化目标，那么究竟该怎么采样能更高效的准确的估计 Advantage

作者首先尝试了 single path，即采样完整轨迹

本质上是使用旧策略跑完整局，这种方案的问题显而易见：

- 等待时间过长
- 每个样本一个生命周期只使用一次，估计不稳定

作者提出，对于某个状态，我们真正想知道的是在状态 s 下执行动作 a 会怎样，因此可以固定一个状态 尝试多个动作，就像打游戏中一个选择节点有多个选择，有经验的玩家会在这时存档，然后来回将所有选项选一遍。如同藤蔓一样在一些节点分支，这就是 **Vine**

具体来说，

1. 先跑一条正常轨迹 τ 得到一系列状态 $s_1, s_2, \dots$
2. 从中挑选出一些状态，对这些状态采样 k 个动作
3. 对每个动作执行一次 rollout 得到 Q 值估计 $\hat{Q}_i(s_n, a_{n,k})$

对于小空间（**有限动作空间**），可以把所有动作都试一遍

$$L_n(\theta) = \sum_{k=1}^K \frac{\pi_\theta(a_k|s_n)}{q(a_k|s_n)} \hat{Q}(s_n, a_k). \quad (13)$$

对于**连续动作空间**，无法全部枚举，于是使用**重要性**采样构造估计器

$$L_n(\theta) = \frac{\sum_{k=1}^K \frac{\pi_\theta(a_{n,k}|s_n)}{\pi_{\theta_{\text{old}}}(a_{n,k}|s_n)} \hat{Q}(s_n, a_{n,k})}{\sum_{k=1}^K \frac{\pi_\theta(a_{n,k}|s_n)}{\pi_{\theta_{\text{old}}}(a_{n,k}|s_n)}}. \quad (14)$$

这个分母进行了归一化很重要，不进行归一化方差很大

TRPO 算法步骤

1. 收集数据

通过 上文提到的 **single path** 或是 **vine** 对数据进行采样

得到 (s, a) 及其对应的 $\hat{Q}(s, a)$

2. 构造目标函数

根据 **公式 (13)** 或 **公式 (14)** 以及 KL 约束 $\bar{D}_{KL} \leq \delta$

用样本平均代替期望 $\hat{L}(\theta)$

3. 求解约束优化

$$\begin{aligned} & \max_{\theta} L(\theta) \\ & s.t. \hat{D}_{KL} \leq \delta \end{aligned}$$

Use conjugate gradient followed by line search

Fisher Matrix

在求解梯度之前我们先要介绍一下 **Fisher Matrix**，因为作者几乎是冷不丁的抛出这个概念

TRPO 真正想限制什么？

根据 **公式 (12)**，作者并非在限制参数 θ 的变化量，而是在限制 $D_{KL}(\pi_{old}, \pi_\theta)$ 即**策略分布变化量**

目前我们无法从参数空间的距离直接观察策略分布的变化，因为即使大小相同的参数变化也会导致完全不同大小的策略变化。

因此，作者想到把 KL 约束写成参数变化的形式

在 θ_{old} 附近展开 KL: $D_{KL}(\theta_{old}, \theta)$

泰勒展开得到

$$D_{KL} \approx D_{KL}(\theta_{old}) + \nabla D_{KL}^T \Delta\theta + \frac{1}{2} \Delta\theta^T H \Delta\theta.$$

由于 $D_{KL}(\theta_{old}, \theta_{old}) = 0$ 且 $\nabla D_{KL} = 0$ ，前两项可以被移除，简化可得

$$D_{KL} \approx \frac{1}{2} \Delta\theta^T H \Delta\theta. \quad (15)$$

KL 的 Hessian 可得

$$H = \nabla^2 D_{KL}$$

作者证明 $H = F$ 即 $F = \nabla^2 D_{KL}(\pi_{old} || \pi_\theta)$ 这个矩阵就是 **Fisher Information Matrix**

外形上看 **公式(15)** 近似于欧式距离

$$||\Delta\theta||^2 = \Delta\theta^T I \Delta\theta$$

只是将 I 替换成了 F ，实际上我们可以理解 Fisher 实际上定义了一种新的距离，只不过不同方向距离的权重因为不同方向的策略敏感度不同而不同

TRPO 局部近似

对目标函数做一阶泰勒展开后

$$L(\theta_{old} + \Delta\theta) \approx L(\theta_{old}) + g^T \Delta\theta$$

因此局部近似问题可以写为

$$\begin{aligned} \max_{\Delta\theta} \quad & g^T \Delta\theta \\ \text{s.t.} \quad & \frac{1}{2} \Delta\theta^T F \Delta\theta \leq \delta \end{aligned}$$

where $\Delta\theta = \theta - \theta_{old}$, $g = \nabla_{\theta} L(\theta) \Big|_{\theta=\theta_{old}}$

构造拉格朗日函数

$$\mathcal{L}(\Delta\theta, \lambda) = g^T \Delta\theta - \lambda \left(\frac{1}{2} \Delta\theta^T F \Delta\theta - \delta \right)$$

对 $\Delta\theta$ 求导

$$\nabla_{\Delta\theta} \mathcal{L} = g - \lambda F \Delta\theta$$

令该项为 0

$$g - \lambda F \Delta\theta = 0$$

简化后有

$$\Delta\theta = \frac{1}{\lambda} F^{-1} g$$

因此

$$\Delta\theta \propto F^{-1} g$$

通俗的总结来说 $F^{-1} g$ 就是 **在不让策略分布变化太大的前提下，能最大提升替代目标的方向**

Discussion

- **需要更好的 Advantage Estimation**

理论假设 $A(s, a)$ 是真值

实际上 $\hat{A}(s, a)$ 误差很大，这个问题后来直接催生了 **GAE**

High-Dimensional Continuous Control Using Generalized Advantage Estimation

- **需要更高效的优化**

TRPO 很贵 每次更新都需要计算 Fisher Matrix -> Conjugate Gradient -> Line Search

这个后来被 **PPO** Proximal Policy Optimization Algorithms 解决

- **理论与函数逼近**

作者认为证明建立在 $A_{\pi_{old}}$ 的准确估计之上，而神经网络是函数逼近器

于是 近似误差 + 采样误差 + 神经网络误差 会不会破坏单调改进？

这个目前强化学习理论仍未能完全解决